CRYPTO HARD

Signing Shadow: Redshift

The challenge setup had 23 Schnorr-style signatures accompanied by 24-bit timestamp values, an AES-GCM ciphertext, and a description talking about clock drift and shadow timing leaks.

The verification equation in `model.py` is standard Schnorr: `g^s = R * pub^e (mod p)` with `e = SHA256(domain || msg || R) mod q`

Spotting the Real Weakness

Before going down a rabbit hole trying to analyze the timestamp stamps or build a hidden number problem (HNP) lattice on the signature nonces, I checked the group parameters in `challenge/data.json` :

```
$ head -n 20 challenge/data.json
```



Group Parameters and Target Logs

- Modulus p is a 161-bit prime (49 decimal digits).
- Subgroup order q is a 160-bit prime.

A 49-digit prime field discrete logarithm is small enough for the **Number Field Sieve (NFS)** to crack directly. CADO-NFS even has an out-of-the-box parameter preset for 50-digit primes (`parameters/dlp/params.p50`). We don't even need the signatures or timestamps; we can just compute the discrete log of pub directly in $\mathbb{F}_p$.

Running CADO-NFS

Fired up `cado-nfs.py` in discrete log mode, targeting both pub and generator g modulo p with subgroup order q:

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

CADO-NFS ran through polynomial selection, sieving, and linear algebra, returning the discrete logs relative to its internal base:

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

Deriving the Key & Decrypting

With both logs in hand, changing base modulo q gives the private key scalar x: `x = (log(pub) * inverse(log(g), q)) % q`

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

Ran the script and got the flag:

```
$ python3 solve.py
```

```
~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Signing_Shadow_Redshift main*
❯ python3 solve.py
[+] Group modulus p: 161-bit prime (vulnerable to NFS DLP)
[+] Subgroup order q: 160-bit prime
[+] Recovered Private Scalar: 0xa13f9648f4f4e416a02f1f0ced932692aca4701
[+] Flag: SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}

~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Signing_Shadow_Redshift main*
❯
```

Solver Execution Proof

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}**